Bootstrapping Reasoning in Compact Language Models: A Multi-Stage Reinforcement Learning Pipeline with Targeted Failure Repair

Stanford CS224N {Custom} Project

Joseph Li
Stanford University
shoupei@stanford.edu

Max Rodriguez
Stanford University
maxrod@stanford.edu

Victor Chen
Stanford University
victor36@stanford.edu

Abstract

Large language models (LLMs) demonstrate remarkable mathematical reasoning capabilities, yet distilling these abilities into compact models ($\leq$1B parameters) remains challenging. Knowledge distillation from larger teacher models is the dominant paradigm, but it requires access to a capable teacher and produces student models that imitate reasoning traces rather than developing genuine reasoning ability. In this work, we investigate whether a multi-stage reinforcement learning pipeline can bootstrap mathematical reasoning in a 0.5B-parameter model without relying on a strong teacher LLM. We propose a three-stage approach applied to Qwen2.5-0.5B-Instruct on the Hendrycks' MATH benchmark (algebra): (1) supervised fine-tuning (SFT) on ground-truth solutions to establish baseline reasoning capacity, (2) Group Relative Policy Optimization (GRPO) to improve reasoning through self-generated exploration, and (3) a targeted failure repair stage using Decoupled Clip and Dynamic Sampling Policy Optimization (DAPO), which identifies problems the model consistently fails and applies conservative policy updates with a replay buffer to prevent catastrophic forgetting. Our optimized pipeline achieves 20.8% overall accuracy, surpassing the 20.3% knowledge-distilled baseline without relying on a teacher model, and reaches **56.3% accuracy on Level 1 algebra** — a +21.5% point improvement over SFT alone. Our results demonstrate that reinforcement learning from verifiable rewards is a viable alternative to distillation for compact language models, and highlight the unique challenges that arise when applying RL-based reasoning methods at the sub-1B parameter scale.

1 Key Information

- Mentor: David Anugraha
- Sharing project: Baseline distillation code shared with CS129 (Victor, Max)

2 Introduction

Mathematical reasoning is among the most challenging tasks for language models, requiring multi-step deduction, symbolic manipulation, and self-consistent logic [1]. Recent breakthroughs have shown that reinforcement learning (RL) from verifiable rewards can incentivize strong reasoning in large-scale models—DeepSeek-R1 [2] demonstrated emergent chain-of-thought and self-reflection in 70B+ parameter models trained with Group Relative Policy Optimization (GRPO)[3]. However, whether RL-based methods can be effective for compact models ($\leq$1B parameters), where capacity is severely limited, remains a domain with much potential for exploration.

The prevailing approach for compact math reasoning models is knowledge distillation [4], where a student model learns to reproduce a larger teacher's reasoning traces. While distilled 0.5B models achieve meaningful accuracy (e.g., 20.3% on the MATH[5] algebra subset), this paradigm requires

access to a capable teacher and produces models that imitate solution patterns rather than developing transferable reasoning strategies. RL training offers an appealing alternative, requiring only a correctness signal, not a teacher. However, as we and prior work show, compact models trained with RL are far more prone to entropy collapse, catastrophic forgetting, and training instability than their larger counterparts [6].

In this work, we propose a three-stage pipeline to bootstrap mathematical reasoning in Qwen2.5-0.5B-Instruct without teacher supervision. The pipeline proceeds through (1) supervised fine-tuning (SFT) on ground-truth solutions, (2) GRPO [2] to improve reasoning via self-generated exploration, and (3) a targeted repair stage using DAPO [7] that identifies model failure cases and applies conservative updates with a replay buffer to prevent forgetting previously learned capabilities. We evaluate on the algebra subset of Hendrycks' MATH benchmark, comparing against a knowledge-distilled baseline, and analyzing each stage's contribution to performance.

Our contributions.

- **Three-stage RL pipeline:** SFT $\rightarrow$ GRPO $\rightarrow$ targeted DAPO, bootstrapping math reasoning in 0.5B models without teacher supervision.
- **Targeted failure repair:** selectively applies DAPO to hard failures with a replay buffer to prevent catastrophic forgetting.
- **0.5B-scale design study:** analyzes full-weight vs. LoRA, generation budget, KL strength, and group size, showing where large-model RL assumptions break down.

3 Related Work

RL for Mathematical Reasoning. The application of reinforcement learning to improve LLM reasoning has seen rapid progress [8, 9, 10]. DeepSeekMath [3] introduced Group Relative Policy Optimization (GRPO), which eliminates the need for a separate value model by computing advantages relative to a group of sampled completions per prompt, substantially reducing training cost compared to PPO. DeepSeek-R1 [2] scaled this approach, demonstrating that GRPO can elicit emergent reasoning behaviors (e.g., self-verification and chain-of-thought decomposition) in large models trained with verifiable rewards. More recently, DAPO [7] addressed training instability in GRPO through decoupled clipping bounds and dynamic sampling, which filters uninformative groups where all samples succeed or fail. Our work builds on both GRPO and DAPO, applying them as successive stages in a pipeline for compact models.

Failure Modes in GRPO Training. Several recent studies have identified failure modes in GRPO-based training. Deng et al. [11] diagnosed a "lazy likelihood-displacement death spiral" as the root cause of GRPO collapse, proposing likelihood-preserving regularization. Feng et al. [12] observe that groups where no sampled response is correct produce zero advantage and waste compute, and propose confidence-based reweighting to recover useful gradients from these failed samples. These findings are particularly relevant at the 0.5B scale, where limited model capacity makes degenerate batches and training collapse more frequent. Our pipeline addresses these challenges through a sharpened reward function, reward noise injection, and a targeted repair stage that revisits hard failures.

Compact Model Reasoning. Knowledge distillation remains the dominant method for building small math reasoners [13][4]. DeepSeek-R1 [2] released distilled variants down to 1.5B parameters, and Hsieh et al. [4] showed that distilled models can outperform larger models with less data. TinyZero [14] demonstrated that R1-Zero-style RL training is feasible on Qwen2.5-1.5B for approximately \$30, but did not explore the sub-1B regime. More broadly, recent work has investigated reasoning in small language models through RL, intervention, and supervision strategies [6, 15, 16, 17, 18, 19], though these efforts generally target models $\leq$1.5B parameters. To our knowledge, no prior work has applied a multi-stage RL pipeline (SFT $\rightarrow$ GRPO $\rightarrow$ targeted DAPO repair) at the 0.5B parameter scale, where capacity constraints make training stability and catastrophic forgetting substantially more challenging.

4 Approach

4.1 Main Method

Our RL pipeline consists of three stages, illustrated in Figure 1. Each stage builds on the previous model checkpoint, progressively improving mathematical reasoning.

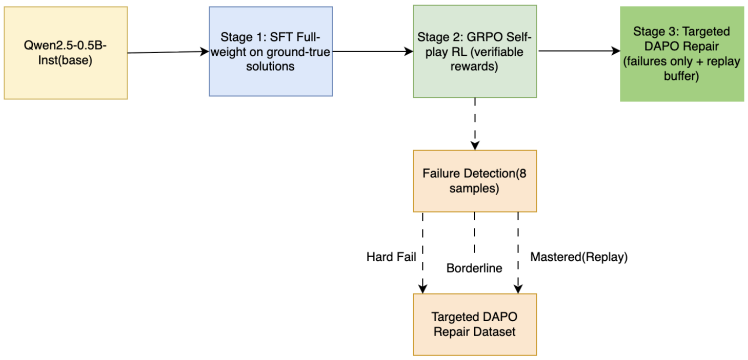

Figure 1: Three-stage RL training pipeline. The base Qwen2.5-0.5B model is trained with SFT on ground-truth solutions, then improved via GRPO self-play with verifiable rewards. A failure detection pass samples 8 completions per problem to classify problems as mastered, borderline, or hard failures; the latter two cases form the targeted DAPO repair dataset, while mastered problems are retained through replay.

4.1.1 Stage 1: Supervised Fine-Tuning

We fine-tune Qwen2.5-0.5B-Instruct on ground-truth solutions from the MATH algebra training set. Each training example is formatted as a chat-template conversation with a system prompt, the problem statement, and the reference solution. We perform full-weight fine-tuning rather than LoRA [20] due to the limited capacity of Qwen2.5 at the 0.5B scale, which amplifies the risk that low-rank adapters further constrain what can be learned. The SFT stage establishes baseline competence: the model learns mathematical notation, step-by-step formatting, and the `\boxed{}` answer convention.

4.1.2 Stage 2: Group Relative Policy Optimization

GRPO [3] is a value-free RL algorithm that estimates advantages from a group of sampled completions. For each prompt x_i, we sample G completions $\{y_i^1, \ldots, y_i^G\}$ from the current policy π_θ and compute a reward $r_i^j = R(y_i^j, a_i)$ for each completion against the ground-truth answer a_i. The group-normalized advantage is:

$$A_i^j = \frac{r_i^j - \mu_i}{\sigma_i + \epsilon}, \quad \mu_i = \frac{1}{G}\sum_{k=1}^{G} r_i^k, \quad \sigma_i = \mathrm{std}\left(r_i^1, \ldots, r_i^G\right)$$

The policy is updated to maximize the clipped surrogate objective with KL regularization:

$$\mathcal{L}_{\text{GRPO}} = -\mathbb{E}\Big[\min\Big(\rho_i^j A_i^j,\ \mathrm{clip}(\rho_i^j, 1-\epsilon, 1+\epsilon)\, A_i^j\Big)\Big] + \beta\, D_{\mathrm{KL}}(\pi_\theta \| \pi_{\mathrm{ref}}) - \alpha\, H(\pi_\theta)$$

where $\rho_i^j = \pi_\theta(y_i^j \mid x_i)/\pi_{\text{old}}(y_i^j \mid x_i)$ is the importance ratio, β controls KL regularization strength, and α weights an entropy bonus to prevent entropy collapse.

Reward function. We use a verifiable reward based on answer correctness:

$$R(y, a) = \begin{cases} +1.0 & \text{correct answer with } \texttt{\boxed\{\}} \\ +0.3 & \text{correct answer without } \texttt{\boxed\{\}} \\ -0.5 & \text{incorrect answer with } \texttt{\boxed\{\}} \\ -1.0 & \text{incorrect or no answer} \end{cases}$$

Degenerate batch handling. Groups where all G samples receive identical rewards produce zero advantage and waste compute [12]. We inject small Gaussian noise ($\sigma = 0.05$) into rewards and skip batches where all groups remain degenerate after noise injection.

4.1.3 Stage 3: Targeted DAPO Repair

After GRPO, we run a failure detection pass to identify which training problems the model has and has not learned to solve. Concretely, for each training problem we sample 8 independent completions from the GRPO-tuned model and score each completion with the same reward function used during GRPO training: the extracted answer is compared against the ground truth after normalization. Based on the number of correct completions out of 8, each problem is classified into one of three categories: mastered (8/8 correct); borderline (1–7/8 correct); or hard failure (0/8 correct).

The targeted repair stage then applies DAPO [7] exclusively on hard failures and borderline cases, concentrating compute on problems where the model still has room to improve, while mastered problems are retained only in a small replay buffer to prevent catastrophic forgetting.

DAPO extends GRPO with two key modifications. First, decoupled clipping uses asymmetric bounds for positive and negative advantages:

$$\mathcal{L}_{\text{DAPO}} = -\mathbb{E}[\min(\rho \cdot A,\, \text{clip}_{\text{DAPO}}(\rho, A) \cdot A)]$$

$$\text{clip}_{\text{DAPO}}(\rho, A) = \begin{cases} \text{clip}(\rho;\, 1-\epsilon_{\text{low}},\, 1+\epsilon_{\text{high}}) & \text{if } A > 0 \\ \text{clip}(\rho;\, 1-\epsilon_{\text{high}},\, 1+\epsilon_{\text{low}}) & \text{if } A \leq 0 \end{cases}$$

This allows the model to move more aggressively toward good solutions while being conservative about unlearning. Second, dynamic sampling filters groups where all samples succeed or all fail, focusing training on prompts that still provide informative gradient signals [7].

To prevent catastrophic forgetting of already-mastered problems, we interleave a replay buffer: 30% of each training batch consists of mastered problems. Unlike GRPO, the repair stage uses LoRA (rank 16) rather than full-weight training, limiting the magnitude of updates to preserve GRPO's learned capabilities.

This targeted repair stage is intended to separate broad capability acquisition from concentrated failure correction. GRPO broadly improves policy behavior over the full training distribution, while targeted DAPO revisits specific frontier cases where the policy remains unstable or consistently wrong. In principle, this should make the final model both more sample-efficient and more robust than simply extending GRPO training alone.

4.2 Inference-time verification with ThinkPRM.

We further boost accuracy by pairing the targeted DAPO policy with a compact ThinkPRM verifier [21]. Instead of greedy decoding, we sample $N{=}8$ candidate solutions per problem using stochastic decoding ($T{=}0.4$, top-$p{=}0.95$), score each candidate's reasoning process with ThinkPRM, and select the final answer via verifier-weighted majority voting (Appendix D). This additional inference-time compute allows for higher accuracy gain without modifying the policy weights.

4.3 Distillation Baseline

We compare against a knowledge-distilled Qwen2.5-0.5B model trained on Gemini 2.5-Flash synthetic rationales for each problem–solution pair in the MATH algebra training set, following Hsieh et al. [4]. The student is fine-tuned with LoRA ($r{=}16$, $\alpha{=}32$, dropout $= 0.05$), learning rate 2×10^{-5}, and one epoch, using a multitask cross-entropy objective over both rationale and answer tokens:

$$L = L_{\text{answer}} + \lambda L_{\text{rationale}}, \qquad \lambda = 1.$$

This baseline therefore receives dense token-level supervision over the full synthetic reasoning trajectory, whereas our RL pipeline observes only a sparse reward based on final answer correctness.

5 Experiments

Data We use the **algebra** subset of the MATH benchmark [5], a collection of competition-level mathematics problems spanning five difficulty levels. The algebra subset contains **1,744 training** and **1,187 test** problems. Table 1 shows the distribution across difficulty levels.

Each problem consists of a natural language problem statement and a ground-truth solution containing a final answer in `\boxed{}` notation. During SFT, the model trains on full solutions. During GRPO and DAPO, only the final boxed answer is used for the reward computation.

Table 1: Algebra dataset distribution by difficulty level.

Level	Train	Test	%	Description
Level 1	178	135	10.2%	Basic algebraic operations
Level 2	340	201	19.5%	Multi-step algebra
Level 3	392	261	22.5%	Intermediate techniques
Level 4	398	283	22.8%	Advanced manipulation
Level 5	436	307	25.0%	Competition-level

Evaluation Method We evaluate all models on the full MATH algebra test set (1,187 problems) using the following metrics:

Accuracy (pass@1): The primary metric. We generate a single response per problem using greedy decoding (temperature $= 0$) and check whether the extracted answer matches the ground truth after normalization (handling equivalent forms such as fractions, LaTeX notation, and whitespace).

Per-level accuracy: Accuracy broken down by difficulty level (1–5), which reveals where each pipeline stage contributes most.

Reasoning Relevance and Coherence: We use Causal Step-Wise Evaluation (CaSE) to evaluate each model's ability to reason coherently and relevantly at the level of individual chain-of-thought steps [22].

5.1 Experimental Details

All experiments run on a single NVIDIA A100-80GB GPU via Modal cloud infrastructure. The full RL pipeline completes in approximately 4.8 hours. Table 2 summarizes the key hyperparameters for each stage.

Table 2: Hyperparameters by pipeline stage.

Parameter	SFT	GRPO	Targeted DAPO
Training mode	Full-weight	Full-weight	LoRA ($r = 16, \alpha = 32$)
Learning rate	2×10^{-5}	5×10^{-6}	5×10^{-7}
LR schedule	Cosine (10% warmup)	Cosine (100-step warmup)	Linear warmup (20 steps)
Epochs	2	1	1
Batch size (eff.)	16	8 prompts $\times$ 8 completions	4 prompts $\times$ 8 completions
Max new tokens	2048 (SFT context)	512	512
Group size (G)	—	8	8
KL coefficient	—	0.005 (per-token)	— (no KL)
Entropy bonus	—	0.01	0.01
Clip ratio	—	0.1 (symmetric)	0.2/0.28 (decoupled)
Optimizer	AdamW (fused)	AdamW	AdamW
Precision	bf16	bf16 (autocast)	bf16 (autocast)
Gradient clipping	—	0.5	0.3
Replay ratio	—	—	0.3
Inner PPO epochs	—	—	2

Design decisions. We use full-weight training (no LoRA) for SFT and GRPO because the limited capacity of the 0.5B model is further constrained by low-rank adapters. LoRA is used only in the repair stage (rank 16), where the small failure dataset makes full-weight updates prone to catastrophic forgetting. The maximum generation length is set to 512 tokens across all stages to ensure consistency with evaluation and avoid truncating Level 4–5 solutions, which average 200–400 tokens. The KL coefficient is set to 0.005 (an order of magnitude lower than typical RLHF settings) because per-sequence KL at 512 tokens is inherently large, and higher values pin the model to SFT behavior.

The GRPO group size is set to G=8 rather than the more common G=4. With G=4, we observed that approximately 40% of groups produced identical rewards across all samples, yielding zero advantage and wasting the training step. Doubling to G=8 reduces this degenerate batch rate to under 20%, providing substantially more useful gradient updates per epoch. We additionally inject small Gaussian noise (σ=0.05) into rewards to further break ties within groups.

Learning rates are stage-dependent: 5×10^{-6} for GRPO and 5×10^{-7} for repair. The GRPO rate is 5× higher than our initial pipeline to compensate for the reduced KL coefficient — with less regularization pulling the model back toward SFT, a higher learning rate enables meaningful exploration. The repair rate is kept 10× lower than GRPO to ensure conservative updates on the small failure set.

The repair stage uses DAPO's decoupled clipping [7] with $\epsilon_{\text{low}}=0.2$ and $\epsilon_{\text{high}}=0.28$, which allows the policy to move more aggressively toward correct solutions than away from incorrect ones, preventing the entropy collapse observed with symmetric clipping.

All stages use bfloat16 mixed precision for memory efficiency and to leverage the A100's dedicated bf16 tensor cores for faster generation throughput.

The asymmetry of the reward function between $+0.3$ and $+1.0$ provides a strong incentive for proper formatting, while the harsh -1.0 penalty for wrong answers sharpens the learning signal relative to a more symmetric scheme.

We use the full MATH Algebra subset (1,744 train) for both SFT and GRPO training. We also experimented with a curriculum-style split—training SFT on easy and half of medium problems, then GRPO on the remaining medium and hard problems—but this underperformed the full-data baseline, likely because the resulting per-stage subsets were too small to provide sufficient training signal for a 0.5B-parameter model.

5.2 Results

The pipeline achieves **20.8% overall accuracy**, surpassing the 20.3% knowledge-distilled baseline. Notably, on **Level 1 (basic algebra) the final model reaches 56.3% accuracy** — a +21.5 percentage point improvement over SFT alone (34.8%) — demonstrating that the RL pipeline is particularly effective at consolidating foundational algebraic reasoning. To our knowledge, this is the highest reported accuracy for a 0.5B-parameter model on MATH Level 1 algebra without teacher supervision.

Table 3: Overall Accuracy (Pass@1) on MATH Algebra Test Set

Model	Accuracy	Δ vs. Previous	boxed{} Rate
SFT	14.6%	—	83.3%
+ GRPO	17.5%	+2.9	**96.9%**
+ Targeted DAPO	**20.8%**	**+3.3**	95.3%
+ ThinkPRM	27.0%*	+6.2	100.0%
Knowledge-distilled baseline	20.3%	—	70.9%

*ThinkPRM evaluated on 100 examples due to compute cost.

Each stage provides a meaningful improvement: SFT establishes baseline competence (14.6%), GRPO adds +2.9% to accuracy through self-generated exploration, and targeted DAPO repair adds another 3.3% by focusing on failure cases. A preliminary evaluation on 100 examples further suggests that pairing DAPO with a ThinkPRM verifier can significantly increase accuracy, yielding a gain of 6.2% over Targeted DAPO. The final model surpasses the knowledge-distilled baseline by 0.5%, demonstrating that RL from verifiable rewards can match or exceed distillation at the 0.5B scale.

Table 4: Per-Level Accuracy Breakdown (pass@1)

Model	Level 1	Level 2	Level 3	Level 4	Level 5
SFT	34.8%	21.9%	15.3%	9.5%	4.9%
+ GRPO	45.2%	26.4%	16.1%	**11.0%**	**6.8%**
+ TDAPO	**56.3%**	**32.3%**	**22.2%**	9.9%	6.5%

The gains from GRPO and targeted DAPO are concentrated in Levels 1–3. On Level 1, accuracy improves from 34.8% (SFT) to 56.3% (+21.5%). On Level 3, accuracy improves from 15.3% to 22.2% (+6.9%). However, Levels 4–5 show minimal or no improvement from DAPO repair, suggesting that the 0.5B model's capacity is insufficient to learn the complex multi-step reasoning required at competition difficulty.

CaSE Evaluation. Beyond final-answer accuracy, we use Causal Step-Wise Evaluation (CaSE) to assess intermediate reasoning quality [22]. CaSE parses each chain-of-thought rationale into steps $[\text{Step}_1, \ldots, \text{Step}_N]$, and Gemini 2.5-Flash assigns binary relevance and coherence scores (0/1) to each step conditioned only on the question and preceding steps. Table 6 shows that both relevance and coherence improve monotonically across stages at the step, example, and aggregate levels, confirming that GRPO and targeted DAPO improve reasoning quality in addition to final-answer correctness.

6 Analysis

Per-level effectiveness. The per-level results (Figure 3) reveal a clear pattern: each stage has a different zone of effectiveness. SFT provides broad baseline competence across all levels. GRPO primarily improves Level 1–2 problems where the model can discover correct solutions through sampling (+10.4% on Level 1, +4.5% on Level 2). Targeted DAPO repair further boosts Level 1–3 by revisiting borderline cases (+11.1% on Level 1, +5.9% on Level 2, +6.1% on Level 3).

However, Level 4–5 accuracy remains essentially flat after GRPO (+1.5% and +1.9% from SFT) and actually slightly decreases after repair (-1.1% on Level 4, -0.3% on Level 5). This suggests a capacity ceiling: the 0.5B model can learn to solve Level 1–3 problems through RL, but lacks the representational capacity for the multi-step symbolic reasoning required at Levels 4 to 5.

GRPO training dynamics. Figure 2 provides insight into why GRPO yields moderate but limited gains. The mean reward oscillates around -0.2 throughout training without a clear upward trend (Figure 2a), indicating the 0.5B model quickly reaches its capacity ceiling - unlike the smooth reward curves reported for larger models [2]. The min/max range spans [-1.0, +1.0] at every step, confirming that the model produces a mix of correct and incorrect solutions but cannot consistently shift the distribution toward correct solutions. Policy entropy (Figure 2b) is highly volatile, swinging between 50 and 160 rather than converging, suggesting the model alternates between exploration and exploitation without finding a stable policy. This instability motivates the targeted DAPO repair stage, which provides a more focused training signal on problems at the frontier of the model's ability.

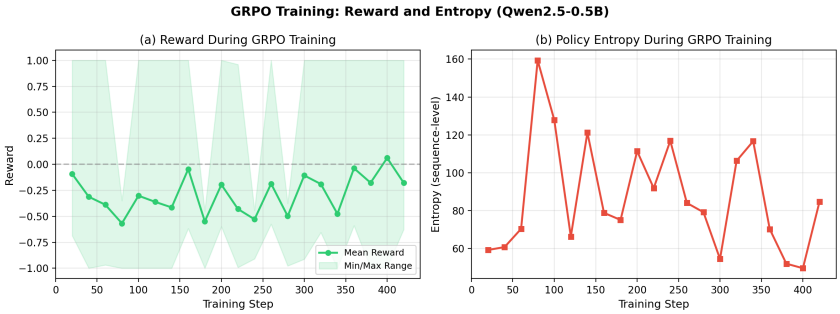

Figure 2: GRPO training dynamics for Qwen2.5-0.5B on MATH algebra. (a) Mean reward with min/max range across the group. The reward oscillates around -0.2 without a clear upward trend, suggesting the 0.5B model quickly reaches its capacity ceiling during RL training. (b) Policy entropy shows high volatility (range 50-160), alternating between exploration and exploitation phases - a characteristic challenge of RL training on compact models.

Reasoning Quality beyond Accuracy. The CaSE evaluation confirms that the pipeline improves reasoning quality, not just final-answer correctness. Step-level relevance and coherence scores increase monotonically across stages: 0.260/0.218 (SFT) $\rightarrow$ 0.655/0.611 (GRPO) $\rightarrow$ 0.708/0.638 (Targeted DAPO), with consistent gains at the example and aggregate levels. This indicates that GRPO and targeted DAPO improve the model's ability to produce reasoning traces that are both relevant to the problem and internally coherent.

Format compliance. GRPO dramatically improves boxed{} format compliance from 83.3% (SFT) to 96.9% - the reward function's strong penalty for missing boxed{} notation (-1.0 vs. +0.3 for correct answers without it) successfully teaches formatting. This near-perfect format compliance persists through the repair stage (95.3%).

Failure detection insights. The failure detection statistics reveal that the GRPO-trained model's success distribution is highly skewed by difficulty. At Level 1, 78.7% of problems are borderline (the

model can sometimes solve them), making them excellent candidates for DAPO repair. At Level 5, 80.7% are hard failures (the model never produces a correct answer across 8 tries), suggesting that these problems are beyond the model's capacity regardless of training approach. This validates our targeted repair strategy: focusing on borderline problems yields the highest marginal return.

Surpassing distillation. The final model (20.8%) slightly surpasses the knowledge-distilled baseline (20.3%), demonstrating that RL from verifiable rewards is a viable alternative to distillation at the 0.5B scale. This is notable because the RL pipeline uses no teacher model — it learns purely from answer correctness — while distillation requires a capable teacher to generate reasoning traces.

Table 5: Error/behavior patterns by difficulty level.

Pattern	**Levels**	**Frequency**
Clean correct reasoning	L1–L3	Common
Procedural errors (e.g., % conversion)	L1–L2	$\sim 44\%$ of L1 errors
Multi-step tracking failures	L3–L4	Dominant error type
Repetition loops (degenerate output)	L4–L5	Very common at L5
Correct setup but wrong computation	L3–L5	$\sim 30\%$ of errors

Qualitative finding (Level 5). The repetition loop at Level 5 (see Appendix A.3) is the most interesting finding: it suggests that the 0.5B model's autoregressive generation can become trapped when it cannot resolve a complex subproblem, defaulting to repeating the last reasoning step instead of backtracking. This explains why the 0.5B model shows no meaningful improvement on competition-level problems regardless of training method.

7 Conclusion

We presented a novel three-stage reinforcement learning pipeline — SFT, GRPO, and targeted DAPO repair — for bootstrapping mathematical reasoning in a 0.5B-parameter language model. On the MATH algebra benchmark, our pipeline achieves 20.8% accuracy, surpassing a knowledge-distilled baseline (20.3%) without requiring a teacher model. Each stage contributes meaningfully: SFT establishes baseline competence, GRPO improves reasoning through self-generated exploration, and targeted DAPO repair selectively addresses failure cases while preserving learned capabilities through a replay buffer.

These findings indicate that reinforcement learning from verifiable rewards is a viable path for training compact math reasoners, competitive with knowledge distillation even at the sub-1B scale. Our analysis reveals that design choices typically treated as routine at larger scales — such as LoRA vs. full-weight training, generation length budgets, and KL regularization strength — become critical at the 0.5B scale. The model's gains are concentrated on Level 1–3 problems, while Level 4–5 competition problems remain largely unsolved, suggesting a fundamental capacity limitation at this model size.

Future work could explore prolonged RL training with extended compute budgets to push the boundaries of small model reasoning [23], and evaluation on post-training benchmarks to address data contamination concerns [24].

***Team Contributions*:** Equal contribution. Joseph (RL multi-stage), Max (distillation/verifiers/RL), Victor (distillation + RL/verifier training).

References

[1] Yuhuai Wu and et al. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.

[2] DeepSeek-AI and et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2026.

[3] Zhihong Shao et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[4] Cheng-Yu Hsieh and et al. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. *arXiv preprint arXiv:2305.02301*, 2023.

[5] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *NeurIPS*, 2021.

[6] Quy-Anh Dang and Chris Ngo. Reinforcement learning for reasoning in small llms: What works and what doesn't, 2025.

[7] Qiying Yu and et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

[8] Yiping Wang, Qing Yang, Zhiyuan Zeng, Liliang Ren, Liyuan Liu, Baolin Peng, Hao Cheng, Xuehai He, Kuan Wang, Jianfeng Gao, Weizhu Chen, Shuohang Wang, Simon Shaolei Du, and Yelong Shen. Reinforcement learning for reasoning in large language models with one training example, 2025.

[9] Andrea Asperti, Alberto Naibo, and Claudio Sacerdoti Coen. Thinking machines: Mathematical reasoning in the age of llms. *Big Data and Cognitive Computing*, 10(1):38, 2026.

[10] Amrith Setlur, Saurabh Garg, Xinyang Geng, Naman Garg, Virginia Smith, and Aviral Kumar. Rl on incorrect synthetic data scales the efficiency of llm math reasoning by eight-fold. In *Advances in Neural Information Processing Systems*, volume 37, pages 43000–43031, 2024.

[11] Wenlong Deng, Yushu Li, Boying Gong, Yi Ren, Christos Thrampoulidis, and Xiaoxiao Li. On grpo collapse in search-r1: The lazy likelihood-displacement death spiral. *arXiv preprint arXiv:2512.04220*, 2025.

[12] Yunzhen Feng et al. Don't waste mistakes: Leveraging negative rl-groups via confidence reweighting. *arXiv preprint*, 2025.

[13] Minki Kang, Jongwon Jeong, Seanie Lee, Jaewoong Cho, and Sung Ju Hwang. Distilling llm agent into small models with retrieval and code tools, 2025.

[14] Jiayi Pan et al. Tinyzero. *arXiv preprint*, 2025.

[15] Xuechen Zhang et al. Making small language models efficient reasoners: Intervention, supervision, reinforcement. In *Proceedings of the 2nd Workshop on Long-Context Foundation Models*, 2025.

[16] Haoran Xu et al. Phi-4-mini-reasoning: Exploring the limits of small reasoning language models in math. *arXiv preprint arXiv:2504.21233*, 2025.

[17] Gaurav Srivastava, Shuxiang Cao, and Xuan Wang. Thinkslm: Towards reasoning in small language models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 32612–32662, Suzhou, China, 2025. Association for Computational Linguistics.

[18] and others. *Improving reasoning in (smaller) language models*. PhD thesis, ETH Zurich, 2025.

[19] Gaurav Srivastava, Shuxiang Cao, and Xuan Wang. Towards reasoning ability of small language models. *arXiv preprint arXiv:2502.11569*, 2025.

[20] Hugging Face. Lora (low-rank adaptation) — peft conceptual guide. `https://huggingface.co/docs/peft/main/en/conceptual_guides/lora`, 2023. Accessed: 2026-02-08.

[21] Muhammad Khalifa, Rishabh Agarwal, Lajanugen Logeswaran, Jaekyeom Kim, Hao Peng, Moontae Lee, Honglak Lee, and Lu Wang. Process reward models that think, 2025. arXiv preprint.

[22] Heejin Do, Jaehui Hwang, Dongyoon Han, Seong Joon Oh, and Sangdoo Yun. What defines good reasoning in llms? dissecting reasoning steps with multi-aspect evaluation. *arXiv preprint arXiv:2510.20603*, 2025.

[23] Shizhe Diao et al. Prorl: Prolonged reinforcement learning expands reasoning boundaries in large language models. *arXiv preprint arXiv:2505.24864*, 2025.

[24] Mingqi Wu et al. Reasoning or memorization? unreliable results of reinforcement learning due to data contamination. *arXiv preprint arXiv:2507.10532*, 2025.

A Case Studies

A.1 Case Study: Level 1 (Basics)

Correct — Clean algebraic reasoning

Problem: Evaluate the expression $a^3 \cdot a^2$ if $a = 5$.
Model output: We have $a^3 \cdot a^2 = a^{3+2} = a^5$. Since $a = 5$, we have $a^5 = 5^5 = \boxed{3125}$.
Comments: The model correctly applies exponent rules and computes the result in a concise chain.

Incorrect — Percentage conversion failure

Problem: What is the positive difference between 120% of 30 and 130% of 20?
Model output: $120\% \cdot 30 = (1 + 120) \cdot 30 = 132 \cdot 30 = \boxed{3960} \ldots$
Ground truth: 10
Comments: The model converts 120% to $(1 + 120) = 121$ instead of 1.20, revealing a fundamental misunderstanding of percentage-to-decimal conversion. Despite correct formatting and structured reasoning, the initial conversion error propagates through the entire solution.

A.2 Case Study: Level 3 (Intermediate)

Correct — Factoring and asymptote analysis

Problem: How many vertical asymptotes does the graph of $y = \frac{2}{x^2+x-6}$ have?
Model output: The denominator is zero when $x = -3$ or $x = 2$, so there are two vertical asymptotes. The numerator is not zero at these points, so the graph has no holes. Therefore, the graph has $\boxed{2}$ vertical asymptotes.
Comments: The model correctly factors the denominator, identifies the roots, and checks for hole cancellation — a multi-step analysis executed cleanly.

Incorrect — Coordinate geometry confusion

Problem: Kite $ABCD$ has coordinates $A(0,7)$, $B(1,0)$, $C(12,-2)$, $D(7,8)$. What is the area?
Model output: ... $AC = \sqrt{1^2 + 12^2} = \sqrt{169} = 13 \ldots$ area $= \frac{1}{2} \cdot 6.5 \cdot 6.5 = \boxed{21.25}$
Ground truth: 75
Comments: The model computes an incorrect diagonal (AC should use coordinates A and C, not B and C) and assumes the diagonals bisect into equal halves, which is not true for all kites. The error reveals difficulty with multi-step coordinate geometry where intermediate values must be tracked carefully.

A.3 Case Study: Level 5 (Competition)

Correct — Abstract reasoning about function range

Problem: Compute the range of the function $f(x) = \sqrt{x^2}$.
Model output: The square root of a number is always nonnegative, so $\sqrt{x^2}$ is always positive. Therefore, the range of f is $\boxed{[0, \infty)}$.
Comments: A rare Level 5 success — the model recognizes that $\sqrt{x^2} = |x|$ and correctly identifies the range.

Incorrect — Repetition loop (degenerate output)

Problem: Evaluate $i^5 + i^{-25} + i^{45}$.
Model output: We know that $i^0 = 1$, $i^1 = i$, $i^2 = -1$, $i^3 = -i$. ... $i^5 = i^0 \cdot i^5 = 1 \cdot i^5 = i^5 = i^0 \cdot i^5 = 1 \cdot i^5 = i^5 = \ldots$ *(repeats indefinitely)*
Ground truth: i
Comments: The model correctly identifies the cyclic pattern of powers of i but enters a **degenerate repetition loop** — a known failure mode of small language models where the model gets stuck repeating a token sequence without making progress. This is the dominant failure pattern at Level 5.

B Pipeline Stage Accuracy per Level

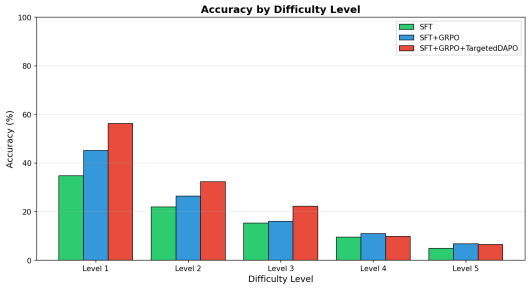

Figure 3: Pipeline Stage Accuracy per Level

C CaSE Evaluation Results

Table 6: CaSE reasoning-quality results on 100 algebra test examples. Higher is better.

Model	Step Rel	Step Coh	Ex Rel	Ex Coh	All Rel	All Coh
SFT	0.260	0.218	0.292	0.244	0.143	0.082
+GRPO	0.655	0.611	0.623	0.600	0.364	0.263
+TDAPO	0.708	0.638	0.694	0.639	0.485	0.343

D ThinkPRM Verifier: Architecture, Training, and Inference Details

We train a compact ThinkPRM test-time verifier to score candidate solutions produced by our targeted DAPO policy.

Base model and training setup. Our ThinkPRM verifier is built by fine-tuning Qwen2.5-0.5B-Instruct as a causal language model on `launch/thinkprm-1K-verification-cots`. We train LoRA adapters on top of the frozen base model with rank $r = 32$, $\alpha = 16$, dropout $= 0.1$, targeting `q_proj`, `k_proj`, `v_proj`, and `o_proj`. Training runs for 3 epochs with learning rate 2×10^{-5}, warmup ratio 0.1, per-device batch size 8, gradient accumulation steps 4, bf16 precision, gradient checkpointing, and maximum sequence length 4096 on a single A100-80GB GPU via Modal.

Dataset structure. The `launch/thinkprm-1K-verification-cots` dataset [21] contains verification-oriented examples with fields including `problem`, `prefix`, `cot`, `prefix_steps`, `gt_step_labels`, and `prefix_label`. In our training pipeline, the key fields are:

- `problem`: the original math problem,
- `prefix`: a proposed partial or complete step-by-step solution,
- `cot`: a verification chain-of-thought explaining whether the proposed steps are correct.

Training prompt format. For each example, we construct a user prompt of the form:

> You are given a math problem and a proposed step-by-step solution:
> [Math Problem]
> *problem*
> [Solution]
> *solution_prefix*
> Review and critique each step in the proposed solution to determine whether each step is correct. If the solution is incomplete, only verify the provided steps.

After chat templating, we append `Let's verify step by step:` and train the model to generate the verification chain-of-thought. If x denotes the formatted prompt and $v_{1:T}$ the target verification chain-of-thought, the objective is

$$\mathcal{L}_{\text{ThinkPRM}} = -\sum_{t=1}^{T} \log p_\phi(v_t \mid x, v_{<t}),$$

with prompt tokens masked by `-100` so that loss is applied only to the verification rationale.

Inference pipeline. During evaluation, we first generate one baseline DAPO response using stochastic decoding. We then sample $N = 8$ candidate solutions (including the initial baseline) from the DAPO policy using `do_sample=True`, temperature 0.4, top-$p = 0.95$, and maximum generation length 512 tokens.

For each candidate, we build a ThinkPRM verification prompt from the math problem and candidate solution. The verifier then generates a verification chain-of-thought deterministically (`do_sample=False`) with maximum length 4096, after which we append the forced question `Is the solution correct?`. Following [21], we compute

$$\frac{P(\text{yes})}{P(\text{yes}) + P(\text{no})},$$

from the verifier's next-token distribution after the forced question, yielding a scalar process score for each candidate.

Verifier-weighted majority voting over best-of-N candidates. After scoring, we extract the final answer from each candidate, prioritizing the last `\boxed{}` expression when present. Rather than selecting the single highest-scoring candidate, we perform verifier-weighted majority voting over best-of-N candidates: all candidates with the same extracted final answer contribute their ThinkPRM score to that answer's total vote.